Facultad de Ingeniería

Carrera de Ingeniería de Sistemas

Implementación de una intranet para la gestión de distribución en una empresa Backus

Avance de Proyecto Final 2

Capa de persistencia y seguridad backend

Desarrollo Web Integrado 100000ST62

Sección 60618

Estudiante

Mathias Sebastian Herrera Gutierrez
Código U23263869

Responsable del análisis, desarrollo y pruebas del APF2

Docente

Cesar Augusto Cardenas la torre

2026 Ciclo 2 Agosto
Lima Perú
09 de octubre de 2026

Índice de contenido

Presentación y continuidad del proyecto 3
Capítulo 1 Configuración y modelo de persistencia 4
1.1 Conexión y entorno de ejecución 4
1.2 Modelo relacional y relaciones normalizadas 5
1.3 Repositorios de Spring Data JPA 7
1.4 Evidencia de la persistencia en PostgreSQL 8
Capítulo 2 Operaciones CRUD JPQL y transacciones 9
2.1 Servicios y operaciones CRUD 9
2.2 Consultas avanzadas con JPQL 10
2.3 Manejo transaccional del inventario 11
2.4 Evidencia de operaciones CRUD 12
Capítulo 3 Seguridad y autenticación JWT 13
3.1 Reglas de acceso y permisos 13
3.2 Generación y validación del token 14
3.3 Filtro de autenticación y vigencia del usuario 15
3.4 Evidencias obligatorias de seguridad 16
Capítulo 4 Pruebas conclusiones y repositorio 17
4.1 Pruebas de endpoints y evidencias 17
4.2 Pruebas de integridad y autorización 18
4.3 Pruebas de integración rollback y concurrencia 19
4.4 Conclusiones técnicas y enlace al proyecto 20
Anexo A Ejecución para la revisión docente 21
Anexo B Colección de Postman y trazabilidad 22
Bibliografía 23

Las figuras y tablas se identifican junto a su contenido. Los anexos explican cómo repetir las pruebas en un entorno local.

Presentación y continuidad del proyecto

El presente avance implementa y verifica la persistencia relacional y la seguridad de la API backend de la intranet de distribución propuesta en el avance 01. El resultado es una API ejecutable con PostgreSQL, operaciones CRUD, consultas JPQL y autenticación JWT, acompañada por pruebas reproducibles y una colección de Postman.

Se conserva el título del caso y la arquitectura por capas del proyecto original. El avance 01 fue elaborado de forma grupal; el APF2 se presenta de manera individual por Mathias Sebastian Herrera Gutierrez, tras el cambio a la sección 60618. La referencia al trabajo de origen se conserva en ORIGEN.md del repositorio, sin atribuir retroactivamente el código compartido a una sola persona.

Backus constituye el contexto del caso académico. Las pruebas usan nombres, direcciones y cuentas sintéticas. El prototipo se valida en un entorno local para evaluar la solución académica propuesta.

Alcance del incremento

Requerimientos del APF1	Resultado entregado
RF 01 a RF 03	Login, roles, permisos y CRUD de usuarios
RF 04 a RF 05	CRUD de sedes y almacenes
RF 06 a RF 07	CRUD de productos y categorías
RF 08	Existencias, entradas, traslados, stock crítico e historial
RF 09 a RF 26	Backlog para avances posteriores

La entrega se concentra en el backend de la Unidad 2. La interfaz Angular, los pedidos, las rutas, la flota y los reportes empresariales permanecen como extensiones del proyecto final. El traslado entre almacenes se incorpora como proceso de negocio para demostrar atomicidad y consistencia del inventario.

Capítulo 1 Configuración y modelo de persistencia

1.1 Conexión y entorno de ejecución

La conexión se configura en application.properties mediante variables de entorno. El arranque local genera una configuración privada, inicia PostgreSQL con Docker Compose y ejecuta Spring Boot. No se incluyen contraseñas ni claves de firma en el repositorio público.

```
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:55432/intranet}
spring.datasource.username=${DB_USERNAME:intranet}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=${DDL_AUTO:update}
spring.jpa.open-in-view=false
spring.jpa.properties.hibernate.dialect=\
  org.hibernate.dialect.PostgreSQLDialect
```

La propiedad ddl-auto=update permite observar la creación automática de tablas requerida por la plantilla. PostgreSQL almacena los registros y aplica claves foráneas, restricciones únicas y comprobaciones de cantidades. La desactivación de open-in-view mantiene la lectura y conversión a DTO dentro de los servicios.

Componente	Versión utilizada	Función
Java	21	Ejecución del backend
Spring Boot	3.5.16	Configuración y API REST
Gradle Wrapper	8.8	Compilación y pruebas reproducibles
PostgreSQL	17.11	Persistencia relacional
JJWT	0.12.6	Firma y validación JWT
Testcontainers	1.21.4	PostgreSQL aislado en pruebas

Se conserva Gradle del repositorio base. La guía menciona Maven como ejemplo de ejecución; el comando equivalente de este proyecto es ./gradlew bootRun. Las versiones y la configuración completa están fijadas en los archivos del repositorio.

1.2 Modelo relacional y relaciones normalizadas

El modelo comprende nueve entidades JPA y una tabla de asociación. Usuario pertenece a un Rol; Rol se vincula con Permiso mediante roles_permisos. En el dominio de inventario, cada Almacén pertenece a una Sede y cada Producto pertenece a una Categoría.

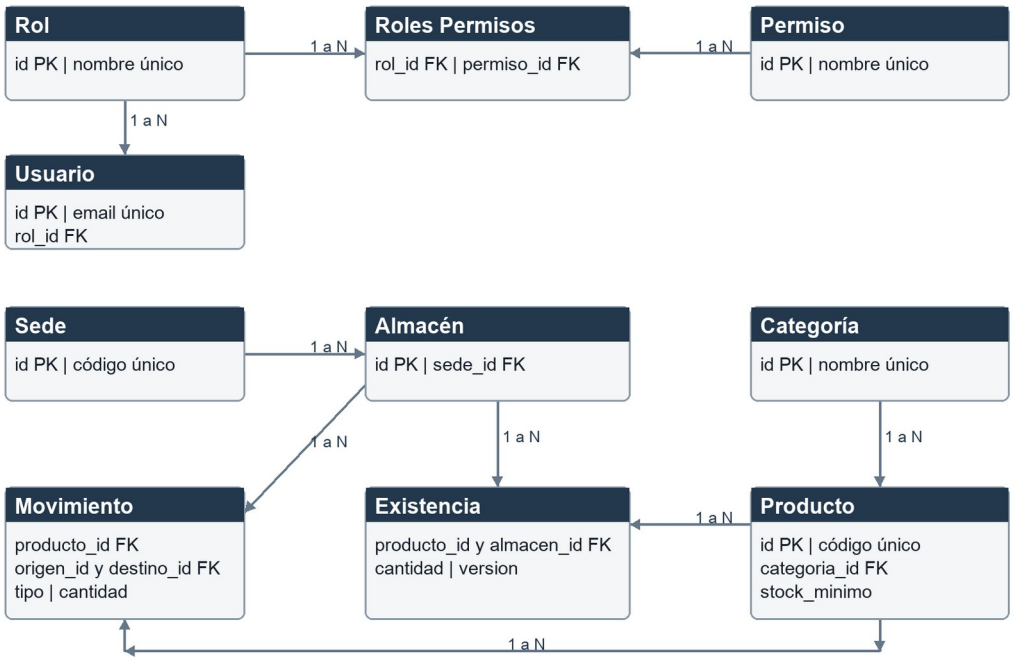


Figura 1 Modelo relacional implementado con entidades JPA y claves foráneas

Relación	Tipo	Mapeo o restricción
Rol a Usuario	1 a N	@ManyToOne en Usuario
Rol a Permiso	N a M	@ManyToMany y roles_permisos
Sede a Almacén	1 a N	@ManyToOne en Almacen
Categoría a Producto	1 a N	@ManyToOne en Producto
Producto y Almacén a Existencia	1 a N por entidad	Par producto_id y almacen_id único
Producto y Almacén a Movimiento	1 a N por entidad	Origen opcional y destino obligatorio

Las relaciones se representan por claves foráneas. La tabla Existencia evita repetir atributos del producto y del almacén; Movimiento conserva el historial de cantidades, origen, destino, fecha y responsable. No se agregan colecciones inversas cuando no existe un caso de uso que las requiera.

1.2 Mapeo ORM de las entidades principales

Las clases del paquete model utilizan @Entity, @Table, @Id y @GeneratedValue. Los identificadores se generan con IDENTITY. El siguiente extracto corresponde al producto y a la existencia por almacén.

```
@Entity
@Table(name="productos")
@Check(constraints="stock_minimo >= 0")
public class Producto {
    @Id @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    @Column(nullable=false, unique=true, length=30)
    private String codigo;
    @Column(name="stock_minimo", nullable=false)
    private Integer stockMinimo;
    @ManyToOne(fetch=FetchType.LAZY, optional=false)
    @JoinColumn(name="categoria_id", nullable=false)
    private Categoria categoria;
}

@Entity
@Table(name="existencias", uniqueConstraints=@UniqueConstraint(
    columnNames={"producto_id","almacen_id"}))
@Check(constraints="cantidad >= 0")
public class Existencia {
    @Id @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    @ManyToOne(fetch=FetchType.LAZY, optional=false)
    @JoinColumn(name="producto_id", nullable=false)
    private Producto producto;
    @ManyToOne(fetch=FetchType.LAZY, optional=false)
    @JoinColumn(name="almacen_id", nullable=false)
    private Almacen almacen;
    @Column(nullable=false) private Integer cantidad = 0;
    @Version private Long version;
}
```

Los extractos omiten getters, setters e importaciones. @Check impide cantidades negativas en PostgreSQL, mientras @Version detecta modificaciones con una versión obsoleta. Los DTO evitan exponer las relaciones completas de Hibernate o el hash de la contraseña en las respuestas de la API.

1.3 Repositorios de Spring Data JPA

Las interfaces del paquete repository heredan de JpaRepository. Sus operaciones save, findById, findAll y delete permiten expresar el CRUD desde el dominio, mientras las consultas específicas se declaran con @Query o métodos derivados.

```
public interface ProductoRepository
        extends JpaRepository<Producto,Long> {
    boolean existsByCategoriaId(Long id);
    // buscar y bloquear se documentan en el capítulo 2
}
```

Repositorio	Entidad	Uso principal
UsuarioRepository	Usuario	CRUD y búsqueda de login con bloqueo
RolRepository	Rol	Roles por ID o nombre
PermisoRepository	Permiso	Permisos válidos y módulos
SedeRepository	Sede	CRUD de sedes
AlmacenRepository	Almacen	CRUD y referencia a sede
CategoriaRepository	Categoria	CRUD de categorías
ProductoRepository	Producto	CRUD, filtros JPQL y bloqueo
ExistenciaRepository	Existencia	Stock por producto y almacén
MovimientoRepository	Movimiento	Historial y referencias existentes

Los servicios resuelven referencias mediante IDs y rechazan asociaciones inexistentes. Antes de eliminar una categoría se comprueba si tiene productos; antes de eliminar una sede se comprueba si tiene almacenes. El inventario y su historial impiden eliminar productos o almacenes que perderían trazabilidad.

Las restricciones de PostgreSQL constituyen una segunda validación ante solicitudes concurrentes. Un código duplicado o una operación que contradice estas relaciones retorna HTTP 409, sin publicar mensajes internos del motor de base de datos.

1.4 Evidencia de la persistencia en PostgreSQL

La siguiente figura reproduce la salida de psql obtenida de la base local después de ejecutar las solicitudes de prueba. Se observan las diez tablas creadas por Hibernate, las existencias y los movimientos almacenados.

PostgreSQL 17 | salida real de psql

Tablas creadas por Hibernate y registros sintéticos después de las pruebas

```
   table_name
----------------
 almacenes
 categorias
 existencias
 movimientos
 permisos
 productos
 roles
 roles_permisos
 sedes
 usuarios
(10 rows)

  codigo  |        nombre        | almacen  | cantidad | stock_minimo
----------+----------------------+----------+----------+--------------
 P-317601 | Producto actualizado | A-317601 |       13 |           10
 P-317601 | Producto actualizado | B-317601 |        7 |           10
(2 rows)

 id |   tipo   | cantidad | origen_id | destino_id |    responsable
----+----------+----------+-----------+------------+--------------------
  1 | ENTRADA  |       20 |           |          1 | admin@example.test
  2 | TRASLADO |        7 |         1 |          2 | admin@example.test
(2 rows)

       email        | es_activo | intentos_fallidos | es_bloqueado
--------------------+-----------+-------------------+--------------
 admin@example.test | t         |                 0 | f
(1 row)
```

Figura 2 Reporte real de PostgreSQL generado con scripts/evidencia-bd.sql

La entrada de 20 unidades y el traslado de 7 unidades producen dos existencias: 13 en el origen y 7 en el destino. Los dos movimientos persistidos permiten reconstruir la operación. El total continúa siendo 20 unidades.

```
docker compose exec -T db psql -U intranet -d intranet -f - \
  < scripts/evidencia-bd.sql
```

Fuente de la evidencia: docs/evidencias/postgresql.txt. Los nombres de los registros son sintéticos y los códigos incluyen un sufijo de prueba. El archivo SQL consulta tablas y registros sin revelar contraseñas o claves de firma.

Capítulo 2 Operaciones CRUD JPQL y transacciones

2.1 Servicios y operaciones CRUD

Se conserva la arquitectura Controller, DTO, Service, implementación y Repository. Los controladores validan entradas con @Valid y comprueban permisos. CatalogoServiceImpl implementa las operaciones del catálogo; UsuarioServiceImpl y RolServiceImpl administran usuarios y roles; InventarioServiceImpl ejecuta entradas y traslados.

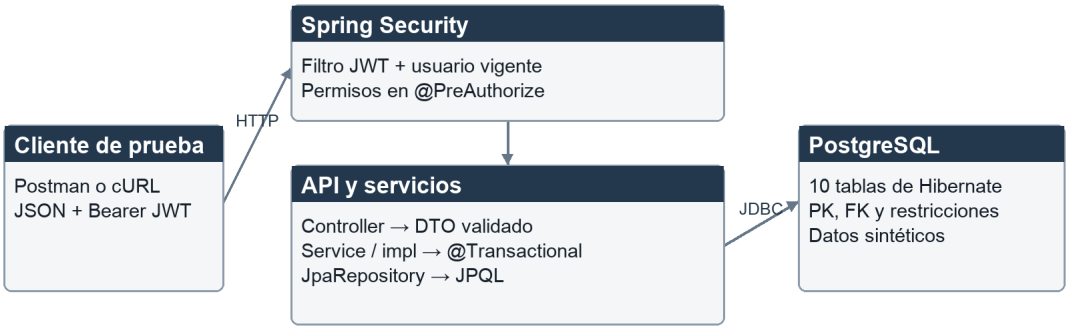


Figura 3 Flujo de una solicitud autenticada hasta la base de datos

```
@Transactional
public ProductoView guardarProducto(Long id, ProductoRequest r) {
    var x = id == null ? new Producto() : obtener(productos,id);
    x.setCodigo(r.codigo().trim().toUpperCase());
    x.setNombre(r.nombre().trim());
    x.setUnidadMedida(r.unidadMedida().trim());
    x.setStockMinimo(r.stockMinimo());
    x.setCategoria(obtener(categorias,r.categoriaId()));
    return ProductoView.from(productos.saveAndFlush(x));
}
```

Operación	Respuesta	Comportamiento
POST	201	Crear un registro validado
GET	200	Consultar datos existentes
PUT	200	Actualizar un registro por ID
DELETE	204	Eliminar cuando no hay referencias
Entrada incorrecta	400 404 409	Validación, ausencia o conflicto

2.2 Consultas avanzadas con JPQL

Se implementan consultas sobre entidades y atributos Java, con parámetros enlazados por @Param. La búsqueda de productos combina categoría y texto sin construir SQL con datos del usuario.

```
@Query("""
    select p from Producto p join fetch p.categoria c
    where (:categoriaId is null or c.id = :categoriaId)
      and (lower(p.nombre) like lower(concat('%', :texto, '%'))
        or lower(p.codigo) like lower(concat('%', :texto, '%')))
    order by p.codigo
    """)
List<Producto> buscar(@Param("categoriaId") Long categoriaId,
                      @Param("texto") String texto);
```

La segunda consulta filtra existencias por almacén y categoría. Al activar critico, devuelve registros cuya cantidad es menor o igual al stock mínimo del producto. JOIN FETCH recupera los datos requeridos para convertir la respuesta a DTO dentro de la transacción de lectura.

```
@Query("""
    select e from Existencia e
    join fetch e.producto p join fetch p.categoria c
    join fetch e.almacen a
    where (:almacenId is null or a.id = :almacenId)
      and (:categoriaId is null or c.id = :categoriaId)
      and (:critico = false or e.cantidad <= p.stockMinimo)
    order by a.codigo, p.codigo
    """)
List<Existencia> buscar(@Param("almacenId") Long almacenId,
    @Param("categoriaId") Long categoriaId,
    @Param("critico") boolean critico);

GET /api/v1/productos?categoriaId=<id>&texto=actualizado
GET /api/v1/inventario?almacenId=<id>&categoriaId=<id>&critico=true
```

Los fragmentos presentan el mismo contenido JPQL con saltos de línea para facilitar la lectura. Los casos 13 y 20 del reporte de cURL comprueban la búsqueda de producto y el stock crítico del almacén destino. Los filtros se encuentran en ProductoRepository y ExistenciaRepository.

2.3 Manejo transaccional del inventario

Una transferencia debe descontar unidades del origen, sumar las mismas unidades al destino y registrar el movimiento. Los tres cambios se ejecutan bajo @Transactional. Si ocurre una excepción, se revierte la unidad completa de trabajo.

```
@Transactional
public MovimientoView trasladar(TrasladoRequest r, String responsable) {
    if (r.origenId().equals(r.destinoId()))
        throw new IllegalArgumentException("Almacenes distintos requeridos");
    Producto producto = bloquear(r.productoId());
    Almacen origen = almacen(r.origenId());
    Existencia salida = stock(producto,origen);
    if (salida.getCantidad() < r.cantidad())
        throw new IllegalStateException("Stock insuficiente");
    salida.setCantidad(salida.getCantidad()-r.cantidad());
    existencias.saveAndFlush(salida);
    Almacen destino = almacen(r.destinoId());
    Existencia llegada = stock(producto,destino);
    llegada.setCantidad(Math.addExact(llegada.getCantidad(),r.cantidad()));
    existencias.save(llegada);
    return registrar(producto,origen,destino,r.cantidad(),
        "TRASLADO",r.motivo(),responsable);
}

@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select p from Producto p where p.id = :id")
Optional<Producto> bloquear(@Param("id") Long id);
```

El bloqueo de la fila del producto serializa sus movimientos hasta finalizar la transacción. Así, dos solicitudes no consumen el mismo stock ni crean dos existencias para el mismo producto y almacén. @Version y las restricciones del esquema añaden protección a la persistencia.

La prueba de rollback usa un destino inexistente después de saveAndFlush de la salida. PostgreSQL recibe una actualización temporal, pero el error revierte el cambio: el origen conserva 13 unidades y no se añade un movimiento. La prueba de concurrencia inicia dos traslados de 7 unidades sobre un origen con 10; uno se confirma y el otro se rechaza.

2.4 Evidencia de operaciones CRUD

Las pruebas HTTP se ejecutaron contra la aplicación iniciada con su PostgreSQL local. La figura reúne creación, lectura y actualización de un producto, junto con la eliminación de otro producto temporal sin referencias.

Operaciones CRUD verificadas por cURL

Reporte de cURL | ejecución local del 09 de octubre de 2026 | secretos omitidos

```
POST /api/v1/productos
Authorization: Bearer [OMITIDO]
HTTP 201   esperado 201   resultado OK
{"id": 1, "codigo": "P-317601", "nombre": "Producto demostrativo", "unidadMedida": "caja",
"stockMinimo": 10, "categoriaId": 1, "categoria": "Bebidas-317601"}

GET /api/v1/productos/1
Authorization: Bearer [OMITIDO]
HTTP 200   esperado 200   resultado OK
{"id": 1, "codigo": "P-317601", "nombre": "Producto demostrativo", "unidadMedida": "caja",
"stockMinimo": 10, "categoriaId": 1, "categoria": "Bebidas-317601"}

PUT /api/v1/productos/1
Authorization: Bearer [OMITIDO]
HTTP 200   esperado 200   resultado OK
{"id": 1, "codigo": "P-317601", "nombre": "Producto actualizado", "unidadMedida": "caja",
"stockMinimo": 10, "categoriaId": 1, "categoria": "Bebidas-317601"}

DELETE /api/v1/productos/3
Authorization: Bearer [OMITIDO]
HTTP 204   esperado 204   resultado OK
""
```

Figura 4 Reporte de solicitudes CRUD reales con sus estados HTTP

El producto del inventario permanece guardado para conservar los movimientos de prueba. La eliminación se comprueba con un producto temporal diferente, seguido por una consulta que devuelve 404. La eliminación de un producto con historial devuelve 409.

Las respuestas completas se conservan en resultados-api.json y resultados-api.md. La guía admite reportes de cURL como evidencia de API. Las figuras de este informe corresponden a exportaciones de los resultados obtenidos mediante cURL. La colección de Postman se entrega para repetir y visualizar las mismas operaciones.

Capítulo 3 Seguridad y autenticación JWT

3.1 Reglas de acceso y permisos

SecurityFilterChain configura una API sin sesiones de servidor. Solo el login y el estado público permiten acceso anónimo. El registro de usuarios exige permiso administrativo para evitar que cualquier visitante asigne roles.

```
.sessionManagement(s ->
    s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(a -> a
    .requestMatchers(HttpMethod.POST,"/api/v1/auth/login").permitAll()
    .requestMatchers(HttpMethod.GET,"/api/v1/public/estado").permitAll()
    .anyRequest().authenticated())
.addFilterBefore(jwtFilter,UsernamePasswordAuthenticationFilter.class)
```

Las decisiones de negocio se expresan con @PreAuthorize. La presencia de un token válido identifica al usuario; sus permisos determinan qué operación puede ejecutar. La denegación por ausencia de autenticación retorna 401 y la falta de permiso retorna 403.

Rol inicial	Lectura	Escritura
ADMIN	Usuarios, roles y catálogo	Administración e inventario
ALMACENERO	Catálogo, stock e historial	Catálogo, entradas y traslados
AUDITOR	Catálogo, stock e historial	Sin permisos de escritura

```
@PreAuthorize("hasAuthority('INVENTARIO_GESTIONAR')")
@PostMapping
public ResponseEntity<ProductoView> crear(
    @Valid @RequestBody ProductoRequest r) { ... }
```

CSRF se desactiva para esta API que recibe el JWT en el encabezado Authorization y no utiliza cookies de sesión. CORS permite los orígenes declarados en CORS_ORIGINS, inicialmente localhost:4200. CORS no sustituye la autenticación ni las reglas de permisos.

3.2 Generación y validación del token

El login valida la contraseña mediante Spring Security y BCrypt. Una autenticación correcta produce un JWT firmado con una SecretKey obtenida de JWT_SECRET, cuyo valor local es aleatorio y está codificado en Base64.

```
private SecretKey key() {
    return Keys.hmacShaKeyFor(Decoders.BASE64.decode(jwtSecret));
}
public String generarToken(Authentication authentication) {
    String username = authentication.getName();
    Date fechaActual = new Date();
    Date fechaExpiracion =
        new Date(fechaActual.getTime()+jwtExpirationDate);
    return Jwts.builder()
        .subject(username)
        .issuedAt(fechaActual)
        .expiration(fechaExpiracion)
        .signWith(key())
        .compact();
}

return Jwts.parser()
    .verifyWith(key())
    .build()
    .parseSignedClaims(token)
    .getPayload()
    .getSubject();
```

JWT_EXPIRATION configura el tiempo de vigencia, con una hora como valor inicial. JJWT selecciona el algoritmo HMAC compatible con el tamaño de la clave; el entorno de demostración generó HS384 con una clave aleatoria de 48 bytes. La prueba registra el algoritmo sin publicar el token.

Un JWT firmado permite comprobar integridad y autenticidad, pero su contenido no está cifrado. En este proyecto contiene el sujeto y las fechas de emisión y expiración; los permisos se recuperan de la base de datos. Una firma alterada, un formato inválido o una fecha expirada impiden el acceso y producen HTTP 401.

Los valores de firma y las contraseñas no se escriben en el código, las evidencias ni la colección exportada. Las respuestas de usuario tampoco incluyen la contraseña o su hash.

3.3 Filtro de autenticación y vigencia del usuario

JwtAuthenticationFilter extiende OncePerRequestFilter. Lee el encabezado Bearer, valida el token y obtiene el usuario vigente. Solo entonces establece Authentication en SecurityContextHolder y permite que continúe la solicitud.

```
String header = req.getHeader("Authorization");
String email = tokens.obtenerUsernameDelJWT(header.substring(7));
var user = users.loadUserByUsername(email);
if (!user.isEnabled() || !user.isAccountNonLocked())
    throw new IllegalArgumentException();
var auth = new UsernamePasswordAuthenticationToken(
    user,null,user.getAuthorities());
SecurityContextHolder.getContext().setAuthentication(auth);
```

El filtro captura JwtException, IllegalArgumentException y UsernameNotFoundException. Así, una firma incorrecta o un usuario eliminado se trata como un acceso no autenticado, en lugar de propagarse como un error 500. El encabezado con un formato distinto a Bearer también se rechaza.

Control de intentos fallidos

El login bloquea la fila de Usuario durante su actualización. Cada contraseña incorrecta incrementa intentosFallidos; el quinto fallo marca la cuenta como bloqueada y devuelve HTTP 423. La transacción usa noRollbackFor para conservar el contador aunque se lance una excepción de autenticación. Un login válido reinicia el contador.

```
@Transactional(noRollbackFor={
    BadCredentialsException.class, LockedException.class})
public JwtAuthResponseDTO login(LoginRequestDTO r) { ... }
```

CustomUserDetailsService refleja el estado esActivo y esBloqueado de la base. Si un usuario se desactiva, se bloquea o se elimina, un token emitido anteriormente deja de habilitar solicitudes. La prueba de integración comprueba la desactivación; la prueba HTTP comprueba el token de un usuario eliminado.

La inscripción del filtro como filtro general del contenedor está desactivada mediante FilterRegistrationBean; su ejecución pertenece a la cadena de Spring Security. Esto conserva un único lugar para aplicar las reglas de la API.

3.4 Evidencias obligatorias de seguridad

Se probó el mismo endpoint protegido antes y después del login. Las tres evidencias muestran ausencia del token, generación exitosa y acceso autorizado. El token se omite al publicar el informe; su generación se comprobó y su valor completo solo existe durante la prueba.

Acceso al catálogo sin token

Reporte de cURL | ejecución local del 09 de octubre de 2026 | secretos omitidos

```
GET /api/v1/productos
Authorization: sin token
HTTP 401   esperado 401   resultado OK
{
  "status": 401,
  "mensaje": "Se requiere un token válido"
}
```

Figura 5 Consulta de productos sin JWT y respuesta HTTP 401

Login exitoso y generación del JWT

Reporte de cURL | ejecución local del 09 de octubre de 2026 | secretos omitidos

```
POST /api/v1/auth/login
Authorization: sin token
HTTP 200   esperado 200   resultado OK
{
  "accessToken": "[OMITIDO]",
  "tokenType": "Bearer"
}
```

Figura 6 Login con respuesta HTTP 200 y accessToken generado y omitido

Acceso al mismo catálogo con JWT válido

Reporte de cURL | ejecución local del 09 de octubre de 2026 | secretos omitidos

```
GET /api/v1/productos
Authorization: Bearer [OMITIDO]
HTTP 200   esperado 200   resultado OK
[]
```

Figura 7 Consulta del mismo endpoint con Bearer y respuesta HTTP 200

Los casos adicionales verifican JWT malformado, firma alterada y permisos insuficientes. La prueba del JWT expirado se ejecuta en la suite de integración. Fuentes: casos 01 a 05 y 27 a 32 del reporte de API, y resultados-integracion.json.

Capítulo 4 Pruebas conclusiones y repositorio

4.1 Pruebas de endpoints y evidencias

El script validar-api.py ejecutó 36 solicitudes con cURL contra el backend local. Todas obtuvieron el estado HTTP esperado. Los casos incluyen respuestas exitosas y rechazos previstos; un 401, 403, 404 o 409 esperado cuenta como validación correcta.

Caso	Operación probada	HTTP	Resultado
01	Acceso protegido sin JWT	401	OK
02	Inicio de sesión y generación JWT	200	OK
03	Mismo endpoint con JWT válido	200	OK
04	JWT malformado	401	OK
05	Firma JWT alterada	401	OK
06	Crear categoría	201	OK
07	Crear sede	201	OK
08	Crear almacén origen	201	OK
09	Crear almacén destino	201	OK
10	Crear producto	201	OK
11	Consultar producto	200	OK
12	Actualizar producto	200	OK
13	Búsqueda JPQL por categoría y nombre	200	OK
14	Rechazar categoría referenciada	409	OK
15	Rechazar código duplicado	409	OK
16	Rechazar datos inválidos	400	OK
17	Entrada de existencias	201	OK
18	Traslado atómico entre almacenes	201	OK

Las evidencias se registraron el 09 de octubre de 2026 en hora de Lima. El script limita su destino a localhost y genera códigos diferentes en cada ejecución. Las contraseñas y los JWT se redactan antes de escribir los archivos de resultados.

4.2 Pruebas de integridad y autorización

Caso	Operación probada	HTTP	Resultado
19	Consultar stock origen después del traslado	200	OK
20	Consulta JPQL de stock crítico	200	OK
21	Rechazar traslado por stock insuficiente	409	OK
22	Rollback con destino inexistente	404	OK
23	Stock conservado tras la reversión	200	OK
24	Historial de movimientos	200	OK
25	Roles y permisos	200	OK
26	Administrador crea auditor	201	OK
27	Inicio de sesión del auditor	200	OK
28	Auditor consulta inventario	200	OK
29	Auditor no puede modificar catálogo	403	OK
30	Auditor no puede administrar usuarios	403	OK
31	Administrador elimina usuario de prueba	204	OK
32	Token de usuario eliminado es rechazado	401	OK
33	Crear producto para comprobar eliminación	201	OK
34	Eliminar producto sin referencias	204	OK
35	Registro eliminado retorna 404	404	OK
36	Rechazar eliminación de producto con historial	409	OK

La prueba de integridad deja el origen con 13 unidades después de un traslado válido. Tanto el rechazo por stock insuficiente como el destino inexistente conservan ese valor. La eliminación de un producto con movimientos se rechaza para conservar el historial.

El rol AUDITOR consulta inventario con HTTP 200, pero la creación de categorías y la administración de usuarios devuelven HTTP 403. Después de eliminar al usuario de prueba, su token devuelve HTTP 401. Estas comprobaciones distinguen autenticación, autorización y vigencia del usuario.

4.3 Pruebas de integración rollback y concurrencia

La suite JUnit utiliza SpringBootTest, MockMvc y PostgreSQL aislado con Testcontainers. Se ejecutaron 14 pruebas: cero fallos, cero errores y cero pruebas omitidas. No se sustituye PostgreSQL por una base en memoria.

Grupo	Validación realizada
CRUD y referencias	Categorías, productos, sedes, almacenes, usuarios y permisos inválidos
JPQL y validaciones	Filtro de producto, stock crítico, duplicidad y datos incorrectos
Autenticación	Ruta pública, ausencia de JWT y registro protegido
JWT	Formato inválido, expiración y firma con otra clave
Permisos y contraseña	Auditor sin escritura y respuestas sin password; BCrypt
Vigencia	Usuario inactivo no reutiliza JWT
Bloqueo de cuenta	Cinco fallos persistidos y bloqueo al quinto intento
Inventario	Conservación de cantidades e historial de dos movimientos
Rollback	Reversión después de enviar la salida a PostgreSQL
Concurrencia	Dos traslados simultáneos no sobregiran stock

```
./gradlew clean test bootJar --no-daemon
Resultado local: BUILD SUCCESSFUL
Pruebas de integración: 14
Fallos: 0 | Errores: 0 | Omitidas: 0
Peticiones cURL: 36 de 36 con el HTTP esperado
```

En la prueba concurrente, el origen inicia con 10 unidades y recibe dos solicitudes de traslado de 7. Solo una se confirma: el origen termina con 3 y el destino con 7. El bloqueo pesimista y la transacción protegen el proceso frente a escrituras simultáneas.

GitHub Actions ejecuta el mismo comando con Java 21 y Docker en cada push. La ejecución 38017439753 finalizó correctamente y publicó el reporte como artefacto del workflow. La evidencia local se encuentra en docs/evidencias/resultados-integracion.json; el resultado completo se genera en build/reports/tests/test/.

4.4 Conclusiones técnicas y enlace al proyecto

Spring Data JPA permite implementar el CRUD mediante contratos de repositorio y entidades relacionadas. Las consultas JPQL expresan filtros del negocio sin depender del nombre físico de las tablas. La integridad se refuerza con validación de DTO, reglas de servicio y restricciones de PostgreSQL.

La transaccionalidad del traslado mantiene como una unidad el descuento, el incremento y el historial. El caso con destino inexistente demuestra rollback después de una escritura efectiva, y el caso concurrente demuestra que dos solicitudes no consumen el mismo stock disponible.

La autenticación JWT protege la identidad de quien invoca la API, mientras los permisos del rol controlan las acciones autorizadas. La consulta del usuario vigente permite rechazar tokens de cuentas inactivas, bloqueadas o eliminadas. BCrypt evita almacenar contraseñas de acceso en texto plano.

El alcance verificado corresponde a persistencia y seguridad backend. Los módulos de pedidos, rutas, flota, reportería y la interfaz de usuario continuarán en posteriores incrementos. Para una puesta en producción se prevén migraciones versionadas, HTTPS, recuperación controlada de cuentas y estrategias de renovación o revocación de tokens.

Repositorio individual público

https://github.com/Deithmho/intranet-distribucion-apf2

El repositorio conserva los paquetes del proyecto base y presenta commits organizados para seguridad, persistencia, pruebas y documentación. ORIGEN.md documenta la procedencia compartida; el APF2 tiene un solo responsable académico.

Commit	Contenido
7baa2a6	Base grupal saneada y procedencia
2d25462	Java 21 y seguridad JWT con permisos
f3d8420	Catálogo JPA y traslados atómicos
9337db8	PostgreSQL, rollback y concurrencia
b0b1fcc	Ejecución, evidencias y Postman

Anexo A Ejecución para la revisión docente

Requisitos previos: JDK 21, Docker con Compose, Git, Bash y OpenSSL. La primera ejecución descarga Gradle, las dependencias Java y la imagen de PostgreSQL. En Windows, los scripts Bash se pueden ejecutar desde Git Bash o WSL con Docker disponible.

1 Clonar e iniciar el backend

```
git clone https://github.com/Deithmho/intranet-distribucion-apf2.git
cd intranet-distribucion-apf2
bash scripts/iniciar.sh
```

El script genera .env sin sobrescribir uno existente, levanta PostgreSQL y ejecuta la aplicación. La API atiende en localhost:8080 y la base en localhost:55432. La contraseña del administrador se consulta en ADMIN_PASSWORD del .env local; no se comparte en el informe.

2 Utilizar una base de datos propia

Como alternativa, generar .env con scripts/configurar-entorno.sh y ajustar DB_URL, DB_USERNAME y DB_PASSWORD con los datos de PostgreSQL propio. application.properties lee esas variables. Antes de ejecutar Gradle, cargar el entorno de la siguiente manera.

```
set -a
source .env
set +a
./gradlew bootRun --no-daemon
```

3 Repetir las pruebas y consultar la base

```
# En otra terminal y dentro del repositorio
./gradlew clean test bootJar --no-daemon
set -a
source .env
set +a
python3 scripts/validar-api.py
docker compose exec -T db psql -U intranet -d intranet -f - \
  < scripts/evidencia-bd.sql
```

Detener Spring Boot con Ctrl+C. docker compose stop db detiene la base y conserva los registros. Para reanudar, ejecutar otra vez scripts/iniciar.sh. La suite Testcontainers usa una base aislada y no modifica la base de desarrollo.

Anexo B Colección de Postman y trazabilidad

Importar postman/APF2.postman_collection.json y APF2.postman_environment.json. Seleccionar APF2 local y completar adminPassword con el valor de ADMIN_PASSWORD del entorno propio. El token y los identificadores se guardan automáticamente durante la ejecución y no están presentes en los archivos exportados.

Ejecutar la colección en orden desde Collection Runner. Se incluyen 22 solicitudes con pruebas de estados HTTP y verificaciones de stock. La colección se ejecutó además con Newman 6.2.3 y sus 24 comprobaciones pasaron. El login inicial almacena token; las operaciones posteriores capturan sedeId, origenId, destinoId, categoriaId y productoId. Un runId permite repetirla con códigos diferentes.

Exigencia de la guía	Evidencia entregada
Conexión y esquema JPA	Capítulo 1; propiedades y reporte PostgreSQL
Entidades y relaciones	Modelo de datos y extractos ORM
JpaRepository y CRUD	Repositorios, servicios y reportes HTTP
Consultas JPQL	Búsquedas por texto, categoría y stock crítico
Transacciones	Traslado, rollback y prueba concurrente
Spring Security	Rutas públicas y matriz de permisos
Generación y filtro JWT	JwtTokenProvider y OncePerRequestFilter
Prueba sin token	Caso 01 con HTTP 401
Login y acceso autorizado	Casos 02 y 03 con HTTP 200
Repositorio y ejecución	URL pública, commits y anexo A

La guía y la plantilla se usan como requisitos del informe. Sus comandos y tecnologías sugeridas se adaptan a Gradle y PostgreSQL, que ya forman parte del proyecto. Las evidencias publicadas ocultan datos de autenticación y conservan rutas, operaciones, estados y resultados suficientes para repetir la revisión.

Bibliografía

SPRING. Spring Boot 3.5 System Requirements [en línea]. Disponible en:

https://docs.spring.io/spring-boot/3.5/system-requirements.html

Consulta: 09 de octubre de 2026.

SPRING. Spring Data JPA Query Methods [en línea]. Disponible en:

https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html

Consulta: 09 de octubre de 2026.

SPRING. Spring Security Username Password Authentication [en línea]. Disponible en:

https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/index.html

Consulta: 09 de octubre de 2026.

JWTK. JJWT Java JWT [en línea]. Disponible en:

https://github.com/jwtk/jjwt

Consulta: 09 de octubre de 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. Transactions [en línea]. Disponible en:

https://www.postgresql.org/docs/17/tutorial-transactions.html

Consulta: 09 de octubre de 2026.

Material académico proporcionado

Avance 01 del proyecto Desarrollo Web Integrado. Informe grupal de planificación y contexto de la intranet de distribución. Documento proporcionado por el estudiante.

GUÍA DE ORIENTACIÓN Y ENTREGABLE semana 10. APF2 Capa de Persistencia y Seguridad Backend. Curso Desarrollo Web Integrado, ciclo 2026 Ciclo 2 Agosto. Material proporcionado para esta evaluación.

PLANTILLA DE INFORME TÉCNICO 2026. Avance de Proyecto Final 2. Esquema de capítulos, evidencias e instrucciones de ejecución. Material proporcionado para esta evaluación.